

Documentação Técnica de Implementação — Smart Save

1. Identidade Visual e Design System

1.1. Tipografia e Hierarquia Visual

A interface da landing page utiliza a família tipográfica **Inter**, importada diretamente através do ecossistema Google Fonts com uma variação de pesos que vai do 300 (Light) ao 900 (Black). Esta escolha de engenharia de design visa estabelecer uma hierarquia visual de forte impacto emocional e legibilidade técnica, permitindo que títulos de grandes dimensões contrastem nitidamente com textos explicativos de leitura fluida, transmitindo ao utilizador a seriedade, urgência e sofisticação do tema abordado.

1.2. Paleta de Cores Narrativa

As cores do projeto não desempenham apenas um papel estético, mas atuam como uma ferramenta de design narrativo. Geridas de forma centralizada através de propriedades customizadas do CSS (:root), a paleta de cores altera a atmosfera do site dinamicamente conforme a navegação do utilizador:

- **Rosa/Roxo Vibrante (--color-brand: #FF007F):** A cor de identidade da marca, aplicada de forma estratégica em botões de conversão (CTAs), logótipo, realces textuais e estados críticos de aviso.
- **Fundo Escuro Opressivo (--color-dark-bg: #0A0A0A):** Configurado no bloco inicial (Hero) e na apresentação do problema, com o intuito de simular a gravidade e o peso das estatísticas de violência de género.
- **Azul Noturno Tecnológico (--color-tech-bg: #080F26):** Entra em vigor no momento em que a solução da plataforma é introduzida, deslocando o foco do utilizador para o ambiente de segurança cibernética e monitoramento em nuvem.
- **Fundo Claro de Liberdade (--color-light-bg: #FFFFFF):** Domina a seção de encerramento e o manifesto institucional, simbolizando o estado final de paz, clareza e liberdade plena que o ecossistema tecnológico visa devolver às vítimas.

2. Engenharia de Front-End e Stack Tecnológica

2.1. Componentização e Build

A aplicação foi estruturada seguindo as melhores práticas modernas de desenvolvimento web, utilizando tecnologias que priorizam a modularidade e o desempenho de execução:

- **React 18.3.1:** Biblioteca baseada em componentes funcionais e reconciliação por árvore virtual (Virtual DOM). É a tecnologia responsável por gerir as transições fluidas de estado da aplicação de forma reativa e instantânea.
- **Vite 5.3.4:** Utilizado como o motor de build e empacotamento (bundler). O Vite substitui ferramentas legadas, oferecendo uma compilação baseada em ES Modules nativos que acelera drasticamente o fluxo de trabalho e otimiza os ficheiros finais de produção.

2.2. Otimização e Desempenho (Zero Dependências de Terceiros)

Para garantir que o carregamento da landing page seja imediato e que a aplicação não sofra com vulnerabilidades ou quebras de pacotes externos, a arquitetura do front-end foi concebida de forma limpa:

- **Ícones em SVG Inline:** Todos os elementos gráficos de suporte (escudo de proteção, coração, símbolos de erro e vetores de radar) foram injetados diretamente no código da aplicação como componentes puros de vetor, eliminando a requisição de ficheiros externos ou fontes de terceiros.
- **Estilização Nativa Pura:** Toda a responsividade e efeitos visuais foram desenvolvidos utilizando CSS nativo estruturado. Isto removeu a necessidade de frameworks utilitários pesados, reduzindo o tamanho total do pacote de dados transferido para o navegador do utilizador.

3. Arquitetura de Lógica Interativa e Comportamental

3.1. Transição Automatizada de Temas via Scroll

A alternância de cores de fundo que acompanha a narrativa do site é executada de forma reativa através de um listener nativo de eventos de rolagem, integrado ao ciclo de vida do React (gancho `useEffect`). O algoritmo calcula em tempo real o rácio de deslocamento vertical (`scrollRatio`) dividindo a posição atual de rolagem pela altura disponível do documento:

- Rácio inferior a 0.20: Ativa o tema escuro original (`theme-dark`).
- Rácio entre 0.20 e 0.65: Transiciona para o tema azul noturno (`theme-tech`).
- Rácio superior a 0.65: Conclui no tema claro (`theme-light`).

A suavidade desta alteração é assegurada por regras de interpolação no CSS (`transition: background-color 1s ease-in-out`), conferindo ao site um aspeto fluido e contínuo.

3.2. Simulador Matemático de Perímetro (Bubble Simulation)

A funcionalidade interativa de simulação da bolha protetiva de 500 metros foi concebida sobre

lógica matemática pura, operando sem a injeção de motores de física externos:

- **Cálculo Dinâmico de Distância:** O sistema captura de forma contínua as coordenadas percentuais (X, Y) do agressor e calcula a distância euclidiana em relação ao ponto central da vítima por meio do Teorema de Pitágoras.
- **Modulação de Estados Ciberfísicos:** O valor resultante é convertido de forma proporcional para uma métrica conceitual de metros. Caso a distância calculada seja inferior ao raio limiar da bolha digital (estabelecido em 28% das dimensões do canvas de simulação), o estado de violação (isViolated) é disparado imediatamente como verdadeiro.
- **Eventos de Ponteiro Unificados:** O simulador mapeia os eventos de movimentação tanto para ambientes desktop via rato (onMouseMove) quanto para dispositivos móveis através de telas táteis (onTouchMove), garantindo uma experiência interativa universal.
- **Disparo Reativo de Alertas:** A alteração do estado lógico injeta instantaneamente classes de erro na interface, modificando o indicador visual de "Área Segura" (verde estável) para "Perímetro Violado" (vermelho pulsante em loop), simulando as notificações que seriam encaminhadas à vítima, à Polícia Militar e ao Poder Judiciário.

3.3. Animações de Elementos (Scroll Reveal)

Para o surgimento gradual dos cartões de dados e textos à medida que o utilizador navega, foi desenvolvida uma lógica de observação geométrica nativa. O script monitoriza o posicionamento dos blocos em relação ao visor de navegação (`getBoundingClientRect().top`). Assim que o topo de um elemento entra na zona correspondente a 85% da altura da janela, o JavaScript adiciona a classe `active`, acionando uma transição de opacidade e translação vertical estruturada em CSS.

4. Infraestrutura de CI/CD e Disponibilidade Contínua (GitHub Actions)

4.1. Desacoplamento de Infraestrutura Local

Para assegurar o correto funcionamento da landing page em ambiente de produção sem exigir qualquer tipo de compilação ou instalação de pacotes no computador local do desenvolvedor, foi desenhada uma esteira automatizada de Integração e Entrega Contínuas (CI/CD) integrada ao ecossistema do GitHub.

4.2. Fluxo de Trabalho Automatizado (deploy.yml)

A esteira é regida por um ficheiro de configuração declarativo em formato YAML, localizado na diretoria oculta do projeto em `.github/workflows/deploy.yml`. O fluxo atua de forma totalmente

autônoma seguindo as etapas técnicas sequenciais:

1. **Gatilho por Evento (Trigger):** O robô monitoriza ativamente o repositório e inicia a execução exclusivamente quando é detectado um envio de atualizações (push) na ramificação de produção (branch main).
2. **Ambiente Técnico Isolado:** O GitHub aloca e inicializa uma máquina virtual temporária rodando a versão estável mais recente do sistema operativo Linux (Ubuntu).
3. **Sincronização e Setup:** O código-fonte é descarregado para a máquina virtual e o ambiente de execução Node.js é estruturado na versão 20.
4. **Instalação Determinística (Clean Install):** O servidor executa o comando de infraestrutura `npm ci`. Este comando analisa o arquivo de bloqueio `package-lock.json` e instala com precisão cirúrgica as mesmas versões de dependências do React e do Vite mapeadas em desenvolvimento, eliminando quaisquer incompatibilidades de compilação.
5. **Processo de Compilação (Build Script):** O comando `npm run build` é acionado. Nesta fase, o compilador do Vite converte todos os módulos isolados, estilos CSS e ficheiros de extensão `.jsx` num pacote puramente estático, minificado e otimizado para o navegador, armazenando o resultado final na diretoria de distribuição `/dist`.
6. **Publicação Direta:** Os arquivos otimizados são extraídos e enviados de forma automática para os servidores de entrega do **GitHub Pages**, e a máquina virtual efêmera é completamente destruída.

4.3. Resiliência, CDN Global e Custo Zero

A entrega final através do GitHub Pages converte o projeto numa aplicação de alto desempenho e manutenção zero. Uma vez que o site é composto apenas de código estático compilado (HTML, CSS e JavaScript tradicionais), ele passa a ser hospedado e distribuído por meio de uma rede de entrega de conteúdo global (CDN). Isto assegura que a landing page institucional do Smart Save permaneça ativa e online 100% do tempo (24/7), apresente velocidades extremas de carregamento em qualquer localização geográfica, possua imunidade a quedas causadas por picos massivos de acessos simultâneos e opere com custo zero de infraestrutura de servidores.

5. Colaboradores do projeto

Gabriel Pierini Dos Santos - 25180000-2

Andre Ruan Silva - 25000814-2

Caio Sobrinho - 25000714-2

Raphael Totti Soller Zacarias - 25000916-2

